
Models

Alejandro Barrachina Argudo (FratosVR)

May 24, 2025

CONTENTS:

1	Introduction	1
2	Installation	3
3	Usage	5
4	Modules	7
	Python Module Index	25
	Index	27

INTRODUCTION

Models is the result of the Thesis of FratosVR. In this repository you can find the code and documentation of the project.

This includes the code to train and use models to detect gestures made by a person using a mocap suit. This are simple models to ensure easy and fast computation.

INSTALLATION

To install the packages needed to run and train the models, you can use the following command:

```
`bash pip install -r requirements.txt `
```


USAGE

Step by step guide on how to train the models:

1. **Download a valid dataset:** ensure that the dataset is from a mocap suit and contains the necessary data for training.
2. **Launch the interface:** run ``python Interface.py`` to open the web interface.
3. **Select the dataset:** in the interface, choose the dataset you downloaded in step 1.
4. **Select the model:** choose the model you want to train from the available options.
5. **Train the model:** click on the “Train” button to start the training process. The interface will show the progress and results of the training.
6. **Evaluate the model:** after training, you can evaluate the model’s performance using the provided metrics in the interface.

You can add new models by creating a new class with the same structure as the existing ones. Make sure to implement the necessary methods and follow the naming conventions used in the project.

MODULES

The modules can be separated into two categories: the ones that are used to train models, and the ones that are models:

4.1 Trainers

- *src.DataLoader*
- *src.Interface*
- *src.Utils*

4.2 Models

- *src.LSTMTrainer*
- *src.CNNTrainer*
- *src.RNNTrainer*
- *src.RandomForestTrainer*

4.2.1 src.DataLoader

Classes

<i>DataLoader</i>	Dataset loader and preprocessor for pose animation data.
-------------------	--

Functions

<i>flatten</i> (xss)	Flatten a nested list structure.
----------------------	----------------------------------

Module Contents

`src.DataLoader.flatten(xss)`

Flatten a nested list structure.

Parameters

xss (*list[list[Any]]*) – Nested list to flatten

Returns

Flattened list

Return type

list[Any]

class src.DataLoader.DataLoader(*dataset_path: str, interval: float = 1.0*)

Dataset loader and preprocessor for pose animation data.

src.DataLoader.__dataset_path: str

src.DataLoader.__interval: float = 1.0

src.DataLoader.__dataset_cleaner_aux(*data: list[tuple[str, str, numpy.ndarray]], max_length: int*) → list[tuple[str, str, numpy.ndarray]]

Clean and standardize animation data to uniform length.

Splits elements exceeding max_length and extends shorter sequences. Removes elements with fewer than 10 frames. Generates new filenames for split sequences with “-1”, “-2” suffixes.

Parameters

- **data** (*list[tuple[str, str, np.ndarray]]*) – Original dataset as list of (label, file-name, array) tuples
- **max_length** (*int*) – Target number of frames per sequence

Returns

Standardized animation sequences

Return type

list[tuple[str, str, np.ndarray]]

src.DataLoader.dataset_cleaner() → None

Clean and standardize the animation dataset.

Processes all CSV files in the dataset directory, ensuring uniform sequence length. Saves cleaned data to ‘./dataset/splitted-animations’ directory.

src.DataLoader.load_dataset_as_dataframe() → pandas.DataFrame

Load and preprocess dataset into a pandas DataFrame.

Returns

DataFrame containing standardized animation sequences with labels

Return type

pd.DataFrame

src.DataLoader.load_dataset() → list[tuple[str, numpy.ndarray]]

Load raw dataset as list of (label, array) tuples.

Returns

List of animation sequences with their category labels

Return type

list[tuple[str, np.ndarray]]

4.2.2 src.Interface

Attributes

interface

Classes

Interface

Interface for training models with Gradio and TensorBoard integration.

Module Contents

class `src.Interface.Interface`

Interface for training models with Gradio and TensorBoard integration.

`src.Interface.log_path: str = None`

List of available models for training

`src.Interface.__models: list[str] = ['LSTM', 'RNN', 'CNN', 'RandomForest']`

Mapping of model names to their respective trainer classes

`src.Interface.__models_map: dict[str, LSTMTrainer.LSTMTrainer | RNNTrainer.RNNTrainer | CNNTrainer.CNNTrainer | RandomForestTrainer.RandomForestTrainer]`

Interval for training in seconds

`src.Interface.__sec_interval = [0.5]`

Gradio Blocks for the user interface

`src.Interface.__blocks`

Initialize the user interface

`src.Interface.__setup_ui()`

Sets up the user interface components using Gradio Blocks.

`src.Interface.__change_interval_all()`

Changes the interval for all models to a predefined set of values.

`src.Interface.__change_interval(value)`

Changes the interval for the selected model to a single value. :param value: The new interval value to set.

`src.Interface.__train(model, path)`

Trains the selected model with the specified dataset path and interval. :param model: The model to train. :param path: The path to the dataset. :return: The best trained model, confusion matrix, and training statistics.

`src.Interface.__refresh_tensorboard(log_path: str) → gradio.update`

Refreshes the TensorBoard interface with the specified log path.

Args:

`log_path (str)`: The path to the TensorBoard logs.

Returns:

`gr.update()`: An update to the Gradio interface to reflect the new TensorBoard state.

`src.Interface.launch()`

Launches the Gradio interface and starts the TensorBoard server.

`src.Interface.interface`

4.2.3 src.Utills

Attributes

<code>customDPI</code>

<code>plot_folder</code>

Functions

<code>plot_confusion_matrix</code> ($\rightarrow$ None)	Generate and save a confusion matrix visualization.
--	---

Module Contents

```
src.Utills.customDPI = 300
```

```
src.Utills.plot_folder = './plots/'
```

```
src.Utills.plot_confusion_matrix(y: numpy.ndarray, p: numpy.ndarray, tags: list[str], filename: str, title: str = "")  $\rightarrow$  None
```

Generate and save a confusion matrix visualization.

Creates a heatmap-style confusion matrix with class labels, grid lines, and value annotations in each cell. Saves the plot to specified file.

Parameters

- **y** (*np.ndarray*) – Ground truth class indices (0-based or 1-based)
- **p** (*np.ndarray*) – Predicted class indices (same indexing as y)
- **tags** (*list[str]*) – Class names corresponding to indices
- **filename** (*str*) – Output file path for saving the plot
- **title** (*str, optional*) – Optional title for the plot, defaults to ""

Note:

- Handles both 0-based and 1-based class indexing automatically
- Uses red color gradient (Reds colormap) for the heatmap
- Includes grid lines between cells for better readability
- Text annotations are white/black for optimal contrast

4.2.4 src.LSTMTrainer

Classes

<code>LSTMTrainer</code>	LSTM Model training manager with hyperparameter tuning using hparams and keras tune.
--------------------------	--

Module Contents

```
class src.LSTMTrainer.LSTMTrainer(interval: str, tensorboard_log_dir: str = './logs/hparams_LTSM')

LSTM Model training manager with hyperparameter tuning using hparams and keras tune.

src.LSTMTrainer.__interval: float

src.LSTMTrainer.__model: tensorflow.keras.Sequential = None

src.LSTMTrainer.__best_model_path: str = 'best_lstm_Uninferable.keras'

src.LSTMTrainer.__cm_file_path: str = None

src.LSTMTrainer.__tensorboard_log_dir: str = './logs/hparams_LTSM'

src.LSTMTrainer.__tensorboard_callbacks: list[tensorflow.keras.callbacks.TensorBoard]

src.LSTMTrainer.__best_acc: float = 0.0

src.LSTMTrainer.__tuner: keras_tuner.Hyperband = None

src.LSTMTrainer.__input_shape: tuple[int, Ellipsis] = None

src.LSTMTrainer.__num_cats: int = None

src.LSTMTrainer.__dropout: float = None

src.LSTMTrainer.__recurrent_dropout: float = None

src.LSTMTrainer.__activation: str = None

src.LSTMTrainer.__recurrent_activation: str = None

src.LSTMTrainer.__unroll: bool = None

src.LSTMTrainer.__use_bias: bool = None

src.LSTMTrainer.__model_generator(input_shape: tuple[int, int], output_shape: int) → None
    LEGACY - Generate a model with the given input and output shape.
```

Parameters

- **input_shape** (*tuple[int, int]*) – Input shape of the model
- **output_shape** (*int*) – Output shape of the model

```
src.LSTMTrainer.train_with_hparams(X: numpy.ndarray, y: numpy.ndarray, X_val: numpy.ndarray = None,
                                   y_val: numpy.ndarray = None, X_test: numpy.ndarray = None, y_test:
                                   numpy.ndarray = None, epochs: int = 10, batch_size: int = 1,
                                   num_cats: int = 6, categories: list[str] = None) → None
```

Train the model with all the combinations of hyperparameters.

Parameters

- **X** (*np.ndarray*) – Input training data
- **y** (*np.ndarray*) – Input training categories
- **X_val** (*np.ndarray, optional*) – Validation input data, defaults to None
- **y_val** (*np.ndarray, optional*) – Validation categories data, defaults to None
- **X_test** (*np.ndarray, optional*) – Test input data, defaults to None

- **y_test** (*np.ndarray*, *optional*) – Test categories data, defaults to None
- **epochs** (*int*, *optional*) – Number of epochs, defaults to 10
- **batch_size** (*int*, *optional*) – Batch size, defaults to 1
- **num_cats** (*int*, *optional*) – Number of categories, defaults to 6
- **categories** (*list[str]*, *optional*) – List of categories, defaults to None

`src.LSTMTrainer.train(X: numpy.ndarray, y: numpy.ndarray, X_val: numpy.ndarray = None, y_val: numpy.ndarray = None, epochs: int = 10, batch_size: int = 1, num_cats: int = 6, categories: list[str] = None) → None`

Train the model with the given parameters.

Parameters

- **X** (*np.ndarray*) – Input training data
- **y** (*np.ndarray*) – Input training categories
- **X_val** (*np.ndarray*, *optional*) – Validation input data, defaults to None
- **y_val** (*np.ndarray*, *optional*) – Validation categories data, defaults to None
- **epochs** (*int*, *optional*) – Number of epochs, defaults to 10
- **batch_size** (*int*, *optional*) – Batch size, defaults to 1
- **num_cats** (*int*, *optional*) – Number of categories, defaults to 6
- **categories** (*list[str]*, *optional*) – List of categories, defaults to None

`src.LSTMTrainer.__build_model(hp: keras_tuner.HyperParameters) → tensorflow.keras.Model`

Build the model with the given hyperparameters.

Parameters

hp (*HyperParameters*) – Hyperparameters (automatically created by keras tuner)

Returns

Compiled Keras model

Return type

`tf.keras.Model`

`src.LSTMTrainer.__update_best_args(acc: float, hparams: dict = None) → None`

Update the best arguments with the new accuracy and time.

Parameters

- **acc** (*float*) – New accuracy value
- **hparams** (*dict*, *optional*) – Hyperparameters dictionary, defaults to None

`src.LSTMTrainer.save_model() → None`

Save the model to a file.

Raises

ValueError – If the model is not initialized

`src.LSTMTrainer.best_model() → str`

Get the best model path.

Returns

Path to the best model

Return type

str

`src.LSTMTrainer.confusion_matrix(filename: str, y_true: numpy.ndarray, y_pred: numpy.ndarray, tags: list[str]) → str`

Generate a confusion matrix for the given model.

Parameters

- **filename** (*str*) – Path to the model file
- **y_true** (*np.ndarray*) – True labels
- **y_pred** (*np.ndarray*) – Predicted labels
- **tags** (*list[str]*) – List of tags

Returns

Path to the confusion matrix image

Return type

str

`src.LSTMTrainer.get_best_acc() → float`

Get the best accuracy.

Returns

Best accuracy value

Return type

float

`src.LSTMTrainer.get_confusion_matrix() → str`

Get the confusion matrix.

Returns

Path to the confusion matrix image

Return type

str

`src.LSTMTrainer.stats() → str`

Get the stats of the model.

Returns

Stats of the model

Return type

str

`src.LSTMTrainer.predict(X: numpy.ndarray) → numpy.ndarray`

Predict the output for the given input.

Parameters

X (*np.ndarray*) – Input data

Returns

Predicted output

Return type

np.ndarray

Raises

ValueError – If the model is not initialized

`src.LSTMTrainer.get_log_dir()` → str

Get the log directory.

Returns

Log directory path

Return type

str

4.2.5 src.CNNTrainer

Classes

<i>CNNTrainer</i>	CNN Model training manager with hyperparameter tuning using hparams and keras tuner.
-------------------	--

Module Contents

class `src.CNNTrainer.CNNTrainer`(*interval: float, tensorboard_log_dir: str = './logs/hparams_CNN'*)

CNN Model training manager with hyperparameter tuning using hparams and keras tuner.

Handles variable-length input sequences for animation classification.

`src.CNNTrainer.__interval:` float

`src.CNNTrainer.__model:` tensorflow.keras.Model = None

`src.CNNTrainer.__best_model_path:` str = 'best_cnn_Uninferable.keras'

`src.CNNTrainer.__tensorboard_log_dir:` str = './logs/hparams_CNN'

`src.CNNTrainer.__tensorboard_callbacks:` list[tensorflow.keras.callbacks.TensorBoard]

`src.CNNTrainer.__best_acc:` float = 0.0

`src.CNNTrainer.__tuner:` keras_tuner.Hyperband = None

`src.CNNTrainer.__input_shape:` tuple = None

`src.CNNTrainer.__num_cats:` int = None

`src.CNNTrainer.__conv_filters:` int = None

`src.CNNTrainer.__kernel_size:` int = None

`src.CNNTrainer.__activation:` str = None

`src.CNNTrainer.__pool_size:` int = None

`src.CNNTrainer.__dense_units:` int = None

`src.CNNTrainer.__dropout_rate:` float = None

`src.CNNTrainer.__model_generator`(*n_features: int, n_classes: int*) → None

LEGACY - Generate a CNN model with given specifications.

Parameters

- **n_features** (*int*) – Number of input features per timestep

- **n_classes** (*int*) – Number of output classes

```
src.CNNTrainer.train_with_hparams(X: numpy.ndarray, y: numpy.ndarray, X_val: numpy.ndarray = None,
                                  y_val: numpy.ndarray = None, X_test: numpy.ndarray = None, y_test:
                                  numpy.ndarray = None, epochs: int = 10, batch_size: int = 1,
                                  num_cats: int = 6, categories: list[str] = None) → None
```

Train model with hyperparameter tuning using Keras Tuner.

Parameters

- **X** (*np.ndarray*) – Training input data
- **y** (*np.ndarray*) – Training target data
- **X_val** (*np.ndarray*, *optional*) – Validation input data, defaults to None
- **y_val** (*np.ndarray*, *optional*) – Validation target data, defaults to None
- **X_test** (*np.ndarray*, *optional*) – Test input data, defaults to None
- **y_test** (*np.ndarray*, *optional*) – Test target data, defaults to None
- **epochs** (*int*, *optional*) – Number of training epochs, defaults to 10
- **batch_size** (*int*, *optional*) – Batch size, defaults to 1
- **num_cats** (*int*, *optional*) – Number of output categories, defaults to 6
- **categories** (*list[str]*, *optional*) – List of category names, defaults to None

```
src.CNNTrainer.train(X: numpy.ndarray, y: numpy.ndarray, X_val: numpy.ndarray = None, y_val:
                    numpy.ndarray = None, epochs: int = 10, batch_size: int = 1, log_dir: str = None,
                    hparams: dict = None) → None
```

Train model with fixed hyperparameters.

Parameters

- **X** (*np.ndarray*) – Training input data
- **y** (*np.ndarray*) – Training target data
- **X_val** (*np.ndarray*, *optional*) – Validation input data, defaults to None
- **y_val** (*np.ndarray*, *optional*) – Validation target data, defaults to None
- **epochs** (*int*, *optional*) – Number of training epochs, defaults to 10
- **batch_size** (*int*, *optional*) – Batch size, defaults to 1
- **log_dir** (*str*, *optional*) – Directory for training logs, defaults to None
- **hparams** (*dict*, *optional*) – Hyperparameters dictionary, defaults to None

```
src.CNNTrainer.__build_model(hp: keras_tuner.HyperParameters) → tensorflow.keras.Model
```

Build model architecture with tunable hyperparameters.

Parameters

hp (*HyperParameters*) – Hyperparameters configuration

Returns

Compiled Keras model

Return type

tf.keras.Model

`src.CNNTrainer.__update_best_args(acc: float, hparams: dict = None) → None`

Update tracking of best model metrics.

Parameters

- **acc** (*float*) – New accuracy value
- **hparams** (*dict, optional*) – Hyperparameters dictionary, defaults to None

`src.CNNTrainer.save_model() → None`

Save current model to disk.

Raises

ValueError – If model is not initialized

`src.CNNTrainer.best_model() → str`

Get path to best saved model.

Returns

Path to model file

Return type

str

`src.CNNTrainer.confusion_matrix(filename: str, y_true: numpy.ndarray, y_pred: numpy.ndarray, tags: list[str]) → str`

Generate and save confusion matrix.

Parameters

- **filename** (*str*) – Model filename
- **y_true** (*np.ndarray*) – True labels
- **y_pred** (*np.ndarray*) – Predicted labels
- **tags** (*list[str]*) – Category names

Returns

Path to saved confusion matrix image

Return type

str

`src.CNNTrainer.get_best_acc() → float`

Get best achieved accuracy.

Returns

Accuracy value

Return type

float

`src.CNNTrainer.get_confusion_matrix() → str`

Get path to confusion matrix image.

Returns

Path to image file

Return type

str

`src.CNNTrainer.stats()` → str

Get training statistics summary.

Returns

Formatted statistics string

Return type

str

`src.CNNTrainer.predict(X: numpy.ndarray)` → numpy.ndarray

Generate predictions for input data.

Parameters

X (*np.ndarray*) – Input data

Returns

Model predictions

Return type

np.ndarray

Raises

ValueError – If model is not initialized

`src.CNNTrainer.get_log_dir()` → str

Get TensorBoard log directory.

Returns

Path to log directory

Return type

str

4.2.6 src.RNNTrainer

Classes

RNNTrainer

RNN Model training manager with hyperparameter tuning using hparams and keras tuner.

Module Contents

class `src.RNNTrainer.RNNTrainer`(*interval: str, tensorboard_log_dir: str = './logs/hparams_RNN'*)

RNN Model training manager with hyperparameter tuning using hparams and keras tuner.

Supports SimpleRNN architecture with comprehensive hyperparameter optimization.

`src.RNNTrainer.__interval:` str

`src.RNNTrainer.__model:` tensorflow.keras.Model = None

`src.RNNTrainer.__best_model_path:` str = 'best_rnn_Uninferable.keras'

`src.RNNTrainer.__tensorboard_log_dir:` str = './logs/hparams_RNN'

`src.RNNTrainer.__tensorboard_callbacks:` list[tensorflow.keras.callbacks.TensorBoard]

`src.RNNTrainer.__best_acc:` float = 0.0

```
src.RNNTrainer.__tuner: keras_tuner.Hyperband = None
src.RNNTrainer.__input_shape: tuple = None
src.RNNTrainer.__num_cats: int = None
src.RNNTrainer.__dropout: float = None
src.RNNTrainer.__recurrent_dropout: float = None
src.RNNTrainer.__activation: str = None
src.RNNTrainer.__unroll: bool = None
src.RNNTrainer.__use_bias: bool = None
src.RNNTrainer.__kernel_initializer: str = None
src.RNNTrainer.__recurrent_initializer: str = None
src.RNNTrainer.__bias_initializer: str = None
src.RNNTrainer.__kernel_regularizer: str = None
src.RNNTrainer.__recurrent_regularizer: str = None
src.RNNTrainer.__bias_regularizer: str = None
src.RNNTrainer.__activity_regularizer: str = None
src.RNNTrainer.__kernel_constraint: str = None
src.RNNTrainer.__recurrent_constraint: str = None
src.RNNTrainer.__bias_constraint: str = None
src.RNNTrainer.__model_generator(input_shape: tuple[int, int], output_shape: int) → None
    LEGACY - Generate RNN model with given specifications.
```

Parameters

- **input_shape** (*tuple[int, int]*) – Input shape (timesteps, features)
- **output_shape** (*int*) – Number of output classes

```
src.RNNTrainer.train_with_hparams(X: numpy.ndarray, y: numpy.ndarray, X_val: numpy.ndarray = None,
                                   y_val: numpy.ndarray = None, X_test: numpy.ndarray = None, y_test:
                                   numpy.ndarray = None, epochs: int = 10, batch_size: int = 1,
                                   num_cats: int = 6, categories: list[str] = None) → None
```

Train model with hyperparameter tuning using Keras Tuner.

Parameters

- **X** (*np.ndarray*) – Training input data
- **y** (*np.ndarray*) – Training target data (one-hot encoded)
- **X_val** (*np.ndarray, optional*) – Validation input data, defaults to None
- **y_val** (*np.ndarray, optional*) – Validation target data, defaults to None
- **X_test** (*np.ndarray, optional*) – Test input data, defaults to None
- **y_test** (*np.ndarray, optional*) – Test target data, defaults to None

- **epochs** (*int, optional*) – Number of training epochs, defaults to 10
- **batch_size** (*int, optional*) – Batch size, defaults to 1
- **num_cats** (*int, optional*) – Number of output categories, defaults to 6
- **categories** (*list[str], optional*) – List of category names, defaults to None

`src.RNNTrainer.train(X: numpy.ndarray, y: numpy.ndarray, X_val: numpy.ndarray = None, y_val: numpy.ndarray = None, epochs: int = 10, batch_size: int = 1, log_dir: str = None, hparams: dict = None) → None`

Train model with fixed hyperparameters.

Parameters

- **X** (*np.ndarray*) – Training input data
- **y** (*np.ndarray*) – Training target data (one-hot encoded)
- **X_val** (*np.ndarray, optional*) – Validation input data, defaults to None
- **y_val** (*np.ndarray, optional*) – Validation target data, defaults to None
- **epochs** (*int, optional*) – Number of training epochs, defaults to 10
- **batch_size** (*int, optional*) – Batch size, defaults to 1
- **log_dir** (*str, optional*) – Directory for training logs, defaults to None
- **hparams** (*dict, optional*) – Hyperparameters dictionary, defaults to None

`src.RNNTrainer.__build_model(hp: keras_tuner.HyperParameters) → tensorflow.keras.Model`

Build model architecture with tunable hyperparameters.

Parameters

hp (*HyperParameters*) – Hyperparameters configuration

Returns

Compiled Keras model

Return type

`tf.keras.Model`

`src.RNNTrainer.__update_best_args(acc: float, hparams: dict = None) → None`

Update tracking of best model metrics.

Parameters

- **acc** (*float*) – New accuracy value
- **hparams** (*dict, optional*) – Hyperparameters dictionary, defaults to None

`src.RNNTrainer.save_model() → None`

Save current model to disk.

Raises

ValueError – If model is not initialized

`src.RNNTrainer.best_model() → str`

Get path to best saved model.

Returns

Path to model file

Return type

`str`

`src.RNNTrainer.confusion_matrix(filename: str, y_true: numpy.ndarray, y_pred: numpy.ndarray, tags: list[str]) → str`

Generate and save confusion matrix plot.

Parameters

- **filename** (*str*) – Model filename base
- **y_true** (*np.ndarray*) – True labels (one-hot encoded)
- **y_pred** (*np.ndarray*) – Predicted labels (one-hot encoded)
- **tags** (*list[str]*) – Category names for labeling

Returns

Path to saved confusion matrix image

Return type

str

`src.RNNTrainer.get_best_acc() → float`

Get best achieved accuracy.

Returns

Accuracy value

Return type

float

`src.RNNTrainer.get_confusion_matrix() → str`

Get path to confusion matrix image.

Returns

Path to image file

Return type

str

`src.RNNTrainer.stats() → str`

Get training statistics summary.

Returns

Formatted statistics string

Return type

str

`src.RNNTrainer.predict(X: numpy.ndarray) → numpy.ndarray`

Generate predictions for input data.

Parameters

X (*np.ndarray*) – Input data

Returns

Model predictions

Return type

np.ndarray

Raises

ValueError – If model is not initialized

`src.RNNTrainer.get_log_dir() → str`

Get TensorBoard log directory.

Returns

Path to log directory

Return type

str

4.2.7 src.RandomForestTrainer

Classes

RandomForestTrainer

Random Forest Model training manager using Yggdrasil Decision Forests.

Module Contents

class `src.RandomForestTrainer.RandomForestTrainer(interval: str, log_dir: str = './RFlog')`

Random Forest Model training manager using Yggdrasil Decision Forests.

Supports hyperparameter tuning and TensorBoard logging for model evaluation.

`src.RandomForestTrainer.__interval: str`

`src.RandomForestTrainer.__model: ydf.RandomForestModel = None`

`src.RandomForestTrainer.__best_model_path: str = './models/best_rf_Uninferable'`

`src.RandomForestTrainer.__cm_file_path: str = None`

`src.RandomForestTrainer.__log_dir: str = './RFlog'`

`src.RandomForestTrainer.__best_accuracy: float = 0.0`

`src.RandomForestTrainer.__tensorboard_callbacks:`

`list[tensorflow.keras.callbacks.TensorBoard]`

`src.RandomForestTrainer.train_with_hparams(X: numpy.ndarray, y: numpy.ndarray, X_val: numpy.ndarray = None, y_val: numpy.ndarray = None, X_test: numpy.ndarray = None, y_test: numpy.ndarray = None, epochs: int = 10, batch_size: int = 1, num_cats: int = 6, categories: list[str] = None) → None`

Train model with hyperparameter tuning using manual search.

Parameters

- **X** (`np.ndarray`) – Training input features
- **y** (`np.ndarray`) – Training target labels (one-hot encoded)
- **X_val** (`np.ndarray`, *optional*) – Validation input features, defaults to None
- **y_val** (`np.ndarray`, *optional*) – Validation target labels, defaults to None
- **X_test** (`np.ndarray`, *optional*) – Test input features, defaults to None
- **y_test** (`np.ndarray`, *optional*) – Test target labels, defaults to None
- **epochs** (`int`, *optional*) – Number of training epochs, defaults to 10

- **batch_size** (*int*, *optional*) – Batch size, defaults to 1
- **num_cats** (*int*, *optional*) – Number of output categories, defaults to 6
- **categories** (*list[str]*, *optional*) – List of category names, defaults to None

`src.RandomForestTrainer.train(X: numpy.ndarray, y: numpy.ndarray, X_val: numpy.ndarray, y_val: numpy.ndarray, categories: list[str] = None) → None`

Train model with default hyperparameters.

Parameters

- **X** (*np.ndarray*) – Training input features
- **y** (*np.ndarray*) – Training target labels (one-hot encoded)
- **X_val** (*np.ndarray*) – Validation input features
- **y_val** (*np.ndarray*) – Validation target labels
- **categories** (*list[str]*, *optional*) – List of category names, defaults to None

`src.RandomForestTrainer.save_model()` → None

Save the current best model to disk.

Raises

ValueError – If model is not initialized

`src.RandomForestTrainer.best_model()` → str

Get path to best saved model.

Returns

Path to model file

Return type

str

`src.RandomForestTrainer.plot_confusion_matrix(filename: str, y_true: numpy.ndarray, y_pred: numpy.ndarray, tags: list[str]) → str`

Generate and save confusion matrix plot.

Parameters

- **filename** (*str*) – Model filename base
- **y_true** (*np.ndarray*) – True labels (one-hot encoded)
- **y_pred** (*np.ndarray*) – Predicted labels (one-hot encoded)
- **tags** (*list[str]*) – Category names for labeling

Returns

Path to saved confusion matrix image

Return type

str

`src.RandomForestTrainer.get_best_acc()` → float

Get best achieved accuracy.

Returns

Best accuracy value

Return type

float

`src.RandomForestTrainer.get_confusion_matrix()` → str

Get path to confusion matrix image.

Returns

Path to image file

Return type

str

`src.RandomForestTrainer.stats()` → str

Get training statistics summary.

Returns

Formatted statistics string

Return type

str

`src.RandomForestTrainer.predict(X: pandas.DataFrame)` → `numpy.ndarray`

Generate predictions for input data.

Parameters

X (*pd.DataFrame*) – Input features DataFrame

Returns

Model predictions

Return type

`np.ndarray`

Raises

ValueError – If model is not initialized

`src.RandomForestTrainer.get_log_dir()` → str

Get TensorBoard log directory.

Returns

Path to log directory

Return type

str

PYTHON MODULE INDEX

S

- `src.CNNTrainer`, [14](#)
- `src.DataLoader`, [7](#)
- `src.Interface`, [8](#)
- `src.LSTMTrainer`, [10](#)
- `src.RandomForestTrainer`, [21](#)
- `src.RNNTrainer`, [17](#)
- `src.Utls`, [10](#)

Symbols

- `__activation` (in module `src.CNNTrainer`), 14
- `__activation` (in module `src.LSTMTrainer`), 11
- `__activation` (in module `src.RNNTrainer`), 18
- `__activity_regularizer` (in module `src.RNNTrainer`), 18
- `__best_acc` (in module `src.CNNTrainer`), 14
- `__best_acc` (in module `src.LSTMTrainer`), 11
- `__best_acc` (in module `src.RNNTrainer`), 17
- `__best_accuracy` (in module `src.RandomForestTrainer`), 21
- `__best_model_path` (in module `src.CNNTrainer`), 14
- `__best_model_path` (in module `src.LSTMTrainer`), 11
- `__best_model_path` (in module `src.RNNTrainer`), 17
- `__best_model_path` (in module `src.RandomForestTrainer`), 21
- `__bias_constraint` (in module `src.RNNTrainer`), 18
- `__bias_initializer` (in module `src.RNNTrainer`), 18
- `__bias_regularizer` (in module `src.RNNTrainer`), 18
- `__blocks` (in module `src.Interface`), 9
- `__build_model()` (in module `src.CNNTrainer`), 15
- `__build_model()` (in module `src.LSTMTrainer`), 12
- `__build_model()` (in module `src.RNNTrainer`), 19
- `__change_interval()` (in module `src.Interface`), 9
- `__change_interval_all()` (in module `src.Interface`), 9
- `__cm_file_path` (in module `src.LSTMTrainer`), 11
- `__cm_file_path` (in module `src.RandomForestTrainer`), 21
- `__conv_filters` (in module `src.CNNTrainer`), 14
- `__dataset_cleaner_aux()` (in module `src.DataLoader`), 8
- `__dataset_path` (in module `src.DataLoader`), 8
- `__dense_units` (in module `src.CNNTrainer`), 14
- `__dropout` (in module `src.LSTMTrainer`), 11
- `__dropout` (in module `src.RNNTrainer`), 18
- `__dropout_rate` (in module `src.CNNTrainer`), 14
- `__input_shape` (in module `src.CNNTrainer`), 14
- `__input_shape` (in module `src.LSTMTrainer`), 11
- `__input_shape` (in module `src.RNNTrainer`), 18
- `__interval` (in module `src.CNNTrainer`), 14
- `__interval` (in module `src.DataLoader`), 8
- `__interval` (in module `src.LSTMTrainer`), 11
- `__interval` (in module `src.RNNTrainer`), 17
- `__interval` (in module `src.RandomForestTrainer`), 21
- `__kernel_constraint` (in module `src.RNNTrainer`), 18
- `__kernel_initializer` (in module `src.RNNTrainer`), 18
- `__kernel_regularizer` (in module `src.RNNTrainer`), 18
- `__kernel_size` (in module `src.CNNTrainer`), 14
- `__log_dir` (in module `src.RandomForestTrainer`), 21
- `__model` (in module `src.CNNTrainer`), 14
- `__model` (in module `src.LSTMTrainer`), 11
- `__model` (in module `src.RNNTrainer`), 17
- `__model` (in module `src.RandomForestTrainer`), 21
- `__model_generator()` (in module `src.CNNTrainer`), 14
- `__model_generator()` (in module `src.LSTMTrainer`), 11
- `__model_generator()` (in module `src.RNNTrainer`), 18
- `__models` (in module `src.Interface`), 9
- `__models_map` (in module `src.Interface`), 9
- `__num_cats` (in module `src.CNNTrainer`), 14
- `__num_cats` (in module `src.LSTMTrainer`), 11
- `__num_cats` (in module `src.RNNTrainer`), 18
- `__pool_size` (in module `src.CNNTrainer`), 14
- `__recurrent_activation` (in module `src.LSTMTrainer`), 11
- `__recurrent_constraint` (in module `src.RNNTrainer`), 18
- `__recurrent_dropout` (in module `src.LSTMTrainer`), 11
- `__recurrent_dropout` (in module `src.RNNTrainer`), 18
- `__recurrent_initializer` (in module `src.RNNTrainer`), 18
- `__recurrent_regularizer` (in module `src.RNNTrainer`), 18
- `__refresh_tensorboard()` (in module `src.Interface`), 9
- `__sec_interval` (in module `src.Interface`), 9
- `__setup_ui()` (in module `src.Interface`), 9
- `__tensorboard_callbacks` (in module `src.CNNTrainer`), 14
- `__tensorboard_callbacks` (in module `src.LSTMTrainer`), 11
- `__tensorboard_callbacks` (in module `src.RNNTrainer`), 18

src.RNNTrainer), 17
 __tensorboard_callbacks (in module *src.RandomForestTrainer*), 21
 __tensorboard_log_dir (in module *src.CNNTrainer*), 14
 __tensorboard_log_dir (in module *src.LSTMTrainer*), 11
 __tensorboard_log_dir (in module *src.RNNTrainer*), 17
 __train() (in module *src.Interface*), 9
 __tuner (in module *src.CNNTrainer*), 14
 __tuner (in module *src.LSTMTrainer*), 11
 __tuner (in module *src.RNNTrainer*), 17
 __unroll (in module *src.LSTMTrainer*), 11
 __unroll (in module *src.RNNTrainer*), 18
 __update_best_args() (in module *src.CNNTrainer*), 15
 __update_best_args() (in module *src.LSTMTrainer*), 12
 __update_best_args() (in module *src.RNNTrainer*), 19
 __use_bias (in module *src.LSTMTrainer*), 11
 __use_bias (in module *src.RNNTrainer*), 18

B

best_model() (in module *src.CNNTrainer*), 16
 best_model() (in module *src.LSTMTrainer*), 12
 best_model() (in module *src.RandomForestTrainer*), 22
 best_model() (in module *src.RNNTrainer*), 19

C

CNNTrainer (class in *src.CNNTrainer*), 14
 confusion_matrix() (in module *src.CNNTrainer*), 16
 confusion_matrix() (in module *src.LSTMTrainer*), 13
 confusion_matrix() (in module *src.RNNTrainer*), 19
 customDPI (in module *src.Utills*), 10

D

DataLoader (class in *src.DataLoader*), 8
 dataset_cleaner() (in module *src.DataLoader*), 8

F

flatten() (in module *src.DataLoader*), 7

G

get_best_acc() (in module *src.CNNTrainer*), 16
 get_best_acc() (in module *src.LSTMTrainer*), 13
 get_best_acc() (in module *src.RandomForestTrainer*), 22
 get_best_acc() (in module *src.RNNTrainer*), 20
 get_confusion_matrix() (in module *src.CNNTrainer*), 16
 get_confusion_matrix() (in module *src.LSTMTrainer*), 13

get_confusion_matrix() (in module *src.RandomForestTrainer*), 22
 get_confusion_matrix() (in module *src.RNNTrainer*), 20
 get_log_dir() (in module *src.CNNTrainer*), 17
 get_log_dir() (in module *src.LSTMTrainer*), 13
 get_log_dir() (in module *src.RandomForestTrainer*), 23
 get_log_dir() (in module *src.RNNTrainer*), 20

I

Interface (class in *src.Interface*), 9
 interface (in module *src.Interface*), 9

L

launch() (in module *src.Interface*), 9
 load_dataset() (in module *src.DataLoader*), 8
 load_dataset_as_dataframe() (in module *src.DataLoader*), 8
 log_path (in module *src.Interface*), 9
 LSTMTrainer (class in *src.LSTMTrainer*), 11

M

module
 src.CNNTrainer, 14
 src.DataLoader, 7
 src.Interface, 8
 src.LSTMTrainer, 10
 src.RandomForestTrainer, 21
 src.RNNTrainer, 17
 src.Utills, 10

P

plot_confusion_matrix() (in module *src.RandomForestTrainer*), 22
 plot_confusion_matrix() (in module *src.Utills*), 10
 plot_folder (in module *src.Utills*), 10
 predict() (in module *src.CNNTrainer*), 17
 predict() (in module *src.LSTMTrainer*), 13
 predict() (in module *src.RandomForestTrainer*), 23
 predict() (in module *src.RNNTrainer*), 20

R

RandomForestTrainer (class in *src.RandomForestTrainer*), 21
 RNNTrainer (class in *src.RNNTrainer*), 17

S

save_model() (in module *src.CNNTrainer*), 16
 save_model() (in module *src.LSTMTrainer*), 12
 save_model() (in module *src.RandomForestTrainer*), 22
 save_model() (in module *src.RNNTrainer*), 19
src.CNNTrainer

- module, 14
- src.DataLoader
 - module, 7
- src.Interface
 - module, 8
- src.LSTMTrainer
 - module, 10
- src.RandomForestTrainer
 - module, 21
- src.RNNTrainer
 - module, 17
- src.Utills
 - module, 10
- stats() (in module src.CNNTrainer), 16
- stats() (in module src.LSTMTrainer), 13
- stats() (in module src.RandomForestTrainer), 23
- stats() (in module src.RNNTrainer), 20

T

- train() (in module src.CNNTrainer), 15
- train() (in module src.LSTMTrainer), 12
- train() (in module src.RandomForestTrainer), 22
- train() (in module src.RNNTrainer), 19
- train_with_hparams() (in module src.CNNTrainer),
15
- train_with_hparams() (in module src.LSTMTrainer),
11
- train_with_hparams() (in module
src.RandomForestTrainer), 21
- train_with_hparams() (in module src.RNNTrainer),
18